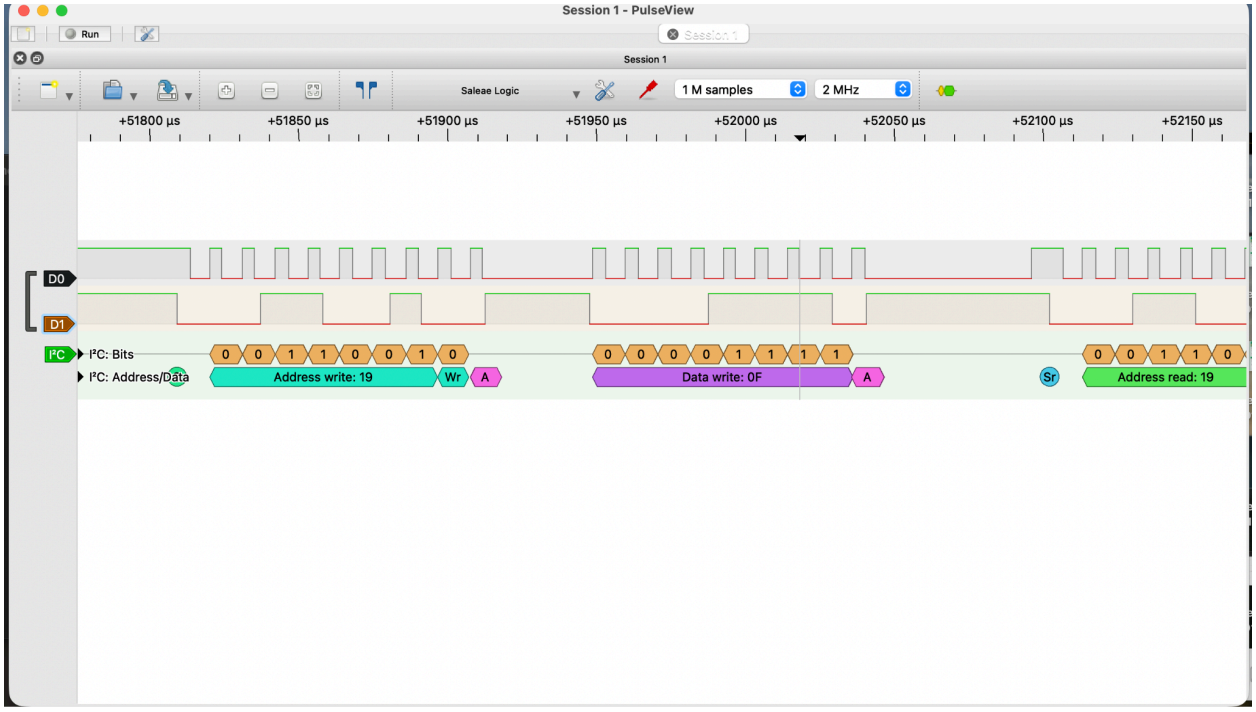
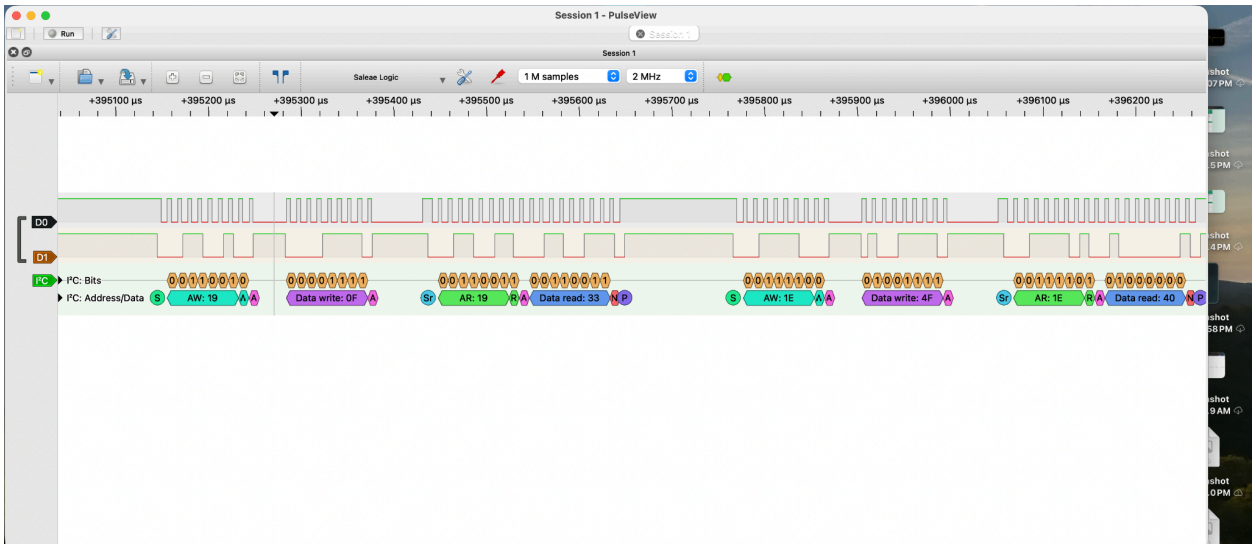
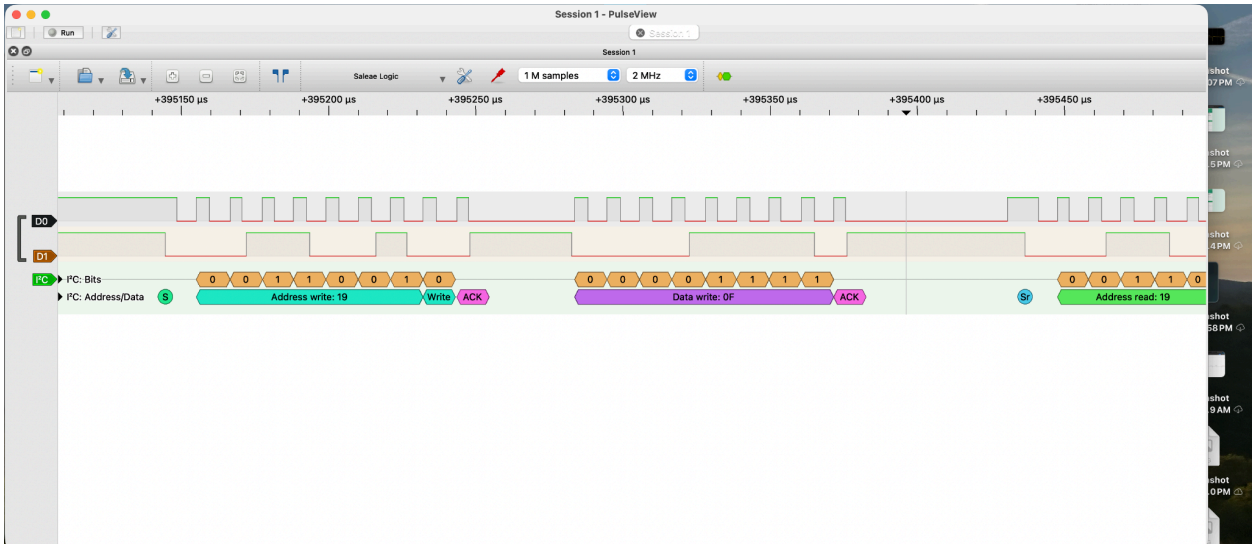
This is the first part: just WHO_AM_I_A

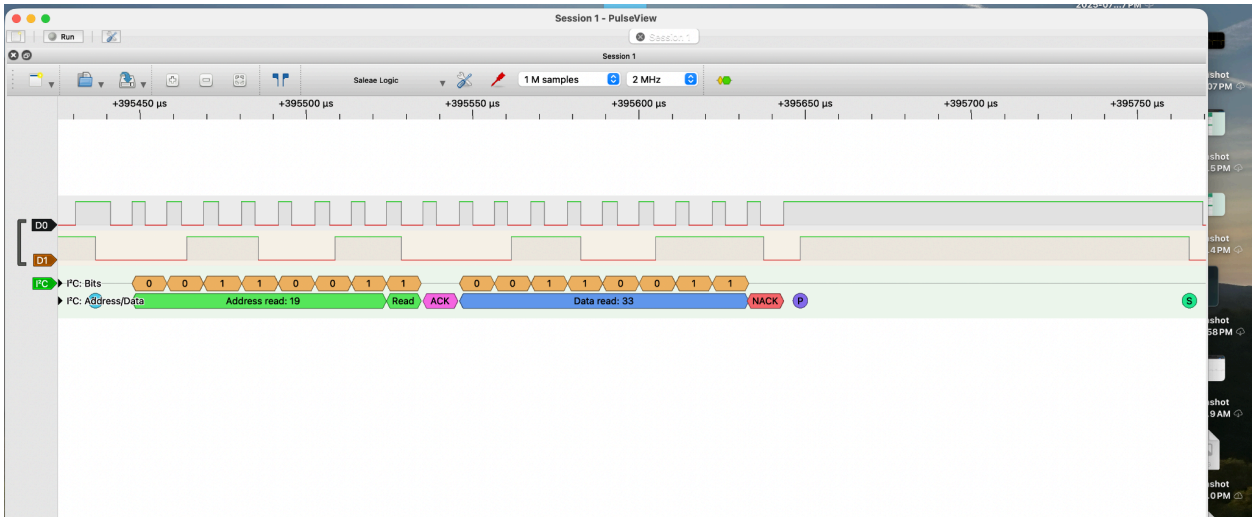


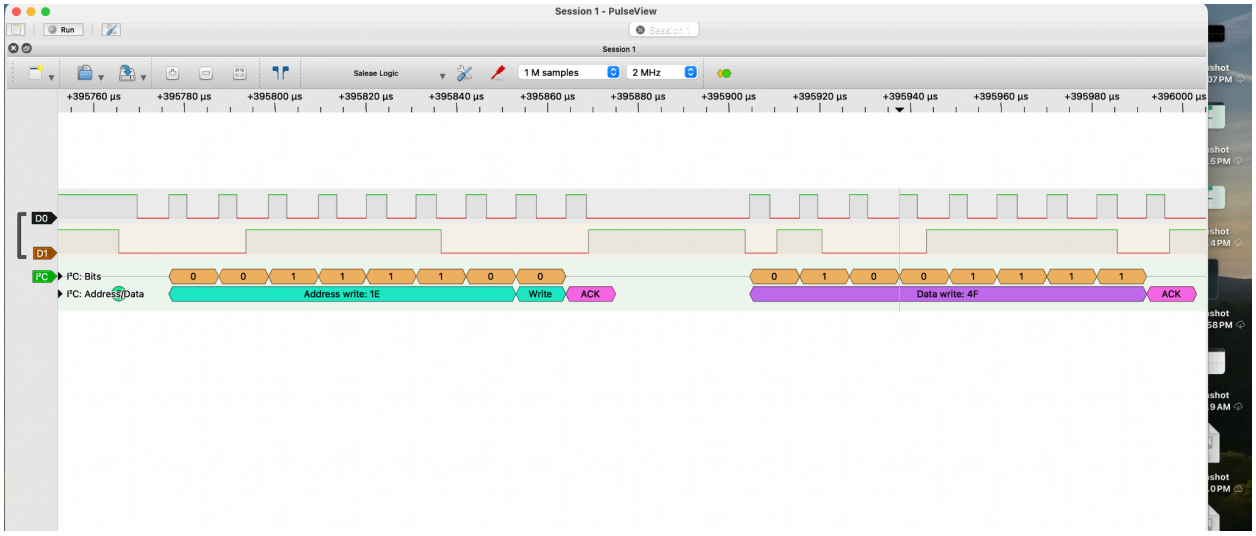
The second part: Both WHO_AM_I_A and WHO_AM_I_M



Zoomed in


Session 1 - PulseView
Run
Session 1
Saleae Logic
1 M samples
2 MHz
+395150 µs
+395200 µs
+395250 µs
+395300 µs
+395350 µs
+395400 µs
+395450 µs
D0
D1
I²C
I²C: Bits
I²C: Address/Data
S
Address write: 19
Write
ACK
Data write: 0F
ACK
Sr
Address read: 19


Session 1 - PulseView
Run
Session 1
Saleae Logic
1 M samples
2 MHz
+395450 µs
+395500 µs
+395550 µs
+395600 µs
+395650 µs
+395700 µs
+395750 µs
D0
D1
I²C
I²C: Bits
I²C: Address/Data
Address read: 19
Read
ACK
Data read: 33
NACK
P
S


Session 1 - PulseView
Run
Session 1
Saleae Logic
1 M samples
2 MHz
+395760 µs
+395780 µs
+395800 µs
+395820 µs
+395840 µs
+395860 µs
+395880 µs
+395900 µs
+395920 µs
+395940 µs
+395960 µs
+395980 µs
+396000 µs
D0
D1
I²C
I²C: Bits
I²C: Address/Data
Address write: 1E
Write
ACK
Data write: 4F
ACK

This is the entire source code, main.c pasted

```
/* USER CODE BEGIN Header */
/**
 ******************************************************************************
 * @file           : main.c
 * @brief          : Main program body
 ******************************************************************************
 * @attention
 *
 * Copyright (c) 2026 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 ******************************************************************************
 */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"
/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include <stdio.h>
/* USER CODE END Includes */
/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */
/* USER CODE END PTD */
/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
/* USER CODE END PD */
/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */
/* USER CODE END PM */
/* Private variables ---------------------------------------------------------*/
I2C_HandleTypeDef hi2c1;
UART_HandleTypeDef hlpuart1;
/* USER CODE BEGIN PV */
/* USER CODE END PV */
/* Private function prototypes -----------------------------------------------*/
```

```c
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_I2C1_Init(void);
static void MX_LPUART1_UART_Init(void);
/* USER CODE BEGIN PFP */
/* USER CODE END PFP */
/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
int __io_putchar(int ch)
{
 HAL_UART_Transmit(&hlpuart1, (uint8_t *)&ch, 1, 10);
 return ch;
}
/* USER CODE END 0 */
/**
 * @brief  The application entry point.
 * @retval int
 */
int main(void)
{
 /* USER CODE BEGIN 1 */
 /* USER CODE END 1 */
 /* MCU Configuration--------------------------------------------------------*/
 /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
 HAL_Init();
 /* USER CODE BEGIN Init */
 /* USER CODE END Init */
 /* Configure the system clock */
 SystemClock_Config();
 /* USER CODE BEGIN SysInit */
 /* USER CODE END SysInit */
 /* Initialize all configured peripherals */
 MX_GPIO_Init();
 MX_I2C1_Init();
 MX_LPUART1_UART_Init();
 /* USER CODE BEGIN 2 */
// printf("\r\nHello from Nucleo-L4R5ZI-P!\r\n");
 uint8_t ctrl1 = 0x37;  // 25 Hz, normal mode, X/Y/Z enabled
 HAL_I2C_Mem_Write(&hi2c1,
                (0x19 << 1),         // accelerometer
```

```
                0x20,               // CTRL_REG1_A
                I2C_MEMADD_SIZE_8BIT,
                &ctrl1,
                1,
                HAL_MAX_DELAY);
  /* USER CODE END 2 */
  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
          uint8_t who_am_i_a = 0;
          uint8_t who_am_i_m = 0;
          HAL_StatusTypeDef status_a = HAL_I2C_Mem_Read(&hi2c1,
                                        (0x19 << 1),        // accelerometer
                                        0x0F,               // WHO_AM_I_A
                                        I2C_MEMADD_SIZE_8BIT,
                                        &who_am_i_a,
                                        1,
                                        HAL_MAX_DELAY);
          HAL_StatusTypeDef status_m = HAL_I2C_Mem_Read(&hi2c1,
                                        (0x1E << 1),        // magnetometer
                                        0x4F,               // WHO_AM_I_M
                                        I2C_MEMADD_SIZE_8BIT,
                                        &who_am_i_m,
                                        1,
                                        HAL_MAX_DELAY);
          printf("A: status=%d WHO_AM_I_A=0x%02X | M: status=%d
WHO_AM_I_M=0x%02X\r\n",
                 status_a, who_am_i_a, status_m, who_am_i_m);
          HAL_Delay(1000);
  }
  /* USER CODE END 3 */
}
/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
```

```
{
 RCC_OscInitTypeDef RCC_OscInitStruct = {0};
 RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
 /** Configure the main internal regulator output voltage
 */
 if (HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1)
!= HAL_OK)
 {
   Error_Handler();
 }
 /** Initializes the RCC Oscillators according to the specified parameters
 * in the RCC_OscInitTypeDef structure.
 */
 RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_MSI;
 RCC_OscInitStruct.MSIState = RCC_MSI_ON;
 RCC_OscInitStruct.MSICalibrationValue = 0;
 RCC_OscInitStruct.MSIClockRange = RCC_MSIRANGE_6;
 RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
 if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
 {
   Error_Handler();
 }
 /** Initializes the CPU, AHB and APB buses clocks
 */
 RCC_ClkInitStruct.ClockType =
RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
 RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_MSI;
 RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
 RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
 RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
 if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
 {
   Error_Handler();
 }
}
/**
 * @brief I2C1 Initialization Function
 * @param None
 * @retval None
```

```
*/
static void MX_I2C1_Init(void)
{
 /* USER CODE BEGIN I2C1_Init 0 */
 /* USER CODE END I2C1_Init 0 */
 /* USER CODE BEGIN I2C1_Init 1 */
 /* USER CODE END I2C1_Init 1 */
 hi2c1.Instance = I2C1;
 hi2c1.Init.Timing = 0x00100D14;
 hi2c1.Init.OwnAddress1 = 0;
 hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
 hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
 hi2c1.Init.OwnAddress2 = 0;
 hi2c1.Init.OwnAddress2Masks = I2C_OA2_NOMASK;
 hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
 hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
 if (HAL_I2C_Init(&hi2c1) != HAL_OK)
 {
   Error_Handler();
 }
 /** Configure Analogue filter
 */
 if (HAL_I2CEx_ConfigAnalogFilter(&hi2c1, I2C_ANALOGFILTER_ENABLE) !=
HAL_OK)
 {
   Error_Handler();
 }
 /** Configure Digital filter
 */
 if (HAL_I2CEx_ConfigDigitalFilter(&hi2c1, 0) != HAL_OK)
 {
   Error_Handler();
 }
 /* USER CODE BEGIN I2C1_Init 2 */
 /* USER CODE END I2C1_Init 2 */
}
/**
 * @brief LPUART1 Initialization Function
 * @param None
 * @retval None
```

```
*/
static void MX_LPUART1_UART_Init(void)
{
 /* USER CODE BEGIN LPUART1_Init 0 */
 /* USER CODE END LPUART1_Init 0 */
 /* USER CODE BEGIN LPUART1_Init 1 */
 /* USER CODE END LPUART1_Init 1 */
 hlpuart1.Instance = LPUART1;
 hlpuart1.Init.BaudRate = 115200;
 hlpuart1.Init.WordLength = UART_WORDLENGTH_8B;
 hlpuart1.Init.StopBits = UART_STOPBITS_1;
 hlpuart1.Init.Parity = UART_PARITY_NONE;
 hlpuart1.Init.Mode = UART_MODE_TX_RX;
 hlpuart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
 hlpuart1.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
 hlpuart1.Init.ClockPrescaler = UART_PRESCALER_DIV1;
 hlpuart1.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
 hlpuart1.FifoMode = UART_FIFOMODE_DISABLE;
 if (HAL_UART_Init(&hlpuart1) != HAL_OK)
 {
   Error_Handler();
 }
 if (HAL_UARTEx_SetTxFifoThreshold(&hlpuart1,
UART_TXFIFO_THRESHOLD_1_8) != HAL_OK)
 {
   Error_Handler();
 }
 if (HAL_UARTEx_SetRxFifoThreshold(&hlpuart1,
UART_RXFIFO_THRESHOLD_1_8) != HAL_OK)
 {
   Error_Handler();
 }
 if (HAL_UARTEx_DisableFifoMode(&hlpuart1) != HAL_OK)
 {
   Error_Handler();
 }
 /* USER CODE BEGIN LPUART1_Init 2 */
 /* USER CODE END LPUART1_Init 2 */
}
/**
```

```c
* @brief GPIO Initialization Function
* @param None
* @retval None
*/
static void MX_GPIO_Init(void)
{
 GPIO_InitTypeDef GPIO_InitStruct = {0};
 /* USER CODE BEGIN MX_GPIO_Init_1 */
 /* USER CODE END MX_GPIO_Init_1 */
 /* GPIO Ports Clock Enable */
 __HAL_RCC_GPIOE_CLK_ENABLE();
 __HAL_RCC_GPIOC_CLK_ENABLE();
 __HAL_RCC_GPIOF_CLK_ENABLE();
 __HAL_RCC_GPIOH_CLK_ENABLE();
 __HAL_RCC_GPIOA_CLK_ENABLE();
 __HAL_RCC_GPIOB_CLK_ENABLE();
 __HAL_RCC_GPIOD_CLK_ENABLE();
 __HAL_RCC_GPIOG_CLK_ENABLE();
 HAL_PWREx_EnableVddIO2();
 /*Configure GPIO pins : PE2 PE3 */
 GPIO_InitStruct.Pin = GPIO_PIN_2|GPIO_PIN_3;
 GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
 GPIO_InitStruct.Pull = GPIO_NOPULL;
 GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
 GPIO_InitStruct.Alternate = GPIO_AF13_SAI1;
 HAL_GPIO_Init(GPIOE, &GPIO_InitStruct);
 /*Configure GPIO pins : PF0 PF1 PF2 */
 GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2;
 GPIO_InitStruct.Mode = GPIO_MODE_AF_OD;
 GPIO_InitStruct.Pull = GPIO_NOPULL;
 GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
 GPIO_InitStruct.Alternate = GPIO_AF4_I2C2;
 HAL_GPIO_Init(GPIOF, &GPIO_InitStruct);
 /*Configure GPIO pin : PF7 */
 GPIO_InitStruct.Pin = GPIO_PIN_7;
 GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
 GPIO_InitStruct.Pull = GPIO_NOPULL;
 GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
 GPIO_InitStruct.Alternate = GPIO_AF13_SAI1;
 HAL_GPIO_Init(GPIOF, &GPIO_InitStruct);
```

```
/*Configure GPIO pins : PC0 PC1 PC2 PC3
                       PC4 PC5 */
GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3
                      |GPIO_PIN_4|GPIO_PIN_5;
GPIO_InitStruct.Mode = GPIO_MODE_ANALOG_ADC_CONTROL;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
/*Configure GPIO pin : PA0 */
GPIO_InitStruct.Pin = GPIO_PIN_0;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.Alternate = GPIO_AF1_TIM2;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
/*Configure GPIO pins : PA1 PA3 */
GPIO_InitStruct.Pin = GPIO_PIN_1|GPIO_PIN_3;
GPIO_InitStruct.Mode = GPIO_MODE_ANALOG_ADC_CONTROL;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
/*Configure GPIO pins : PA4 PA5 PA6 PA7 */
GPIO_InitStruct.Pin = GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
GPIO_InitStruct.Alternate = GPIO_AF5_SPI1;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
/*Configure GPIO pin : PB0 */
GPIO_InitStruct.Pin = GPIO_PIN_0;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.Alternate = GPIO_AF2_TIM3;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
/*Configure GPIO pin : PB1 */
GPIO_InitStruct.Pin = GPIO_PIN_1;
GPIO_InitStruct.Mode = GPIO_MODE_ANALOG_ADC_CONTROL;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
/*Configure GPIO pins : PB2 PB6 */
GPIO_InitStruct.Pin = GPIO_PIN_2|GPIO_PIN_6;
```

```
GPIO_InitStruct.Mode = GPIO_MODE_ANALOG;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
/*Configure GPIO pins : PE7 PE8 PE9 PE10
                       PE11 PE12 PE13 */
GPIO_InitStruct.Pin = GPIO_PIN_7|GPIO_PIN_8|GPIO_PIN_9|GPIO_PIN_10
                      |GPIO_PIN_11|GPIO_PIN_12|GPIO_PIN_13;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.Alternate = GPIO_AF1_TIM1;
HAL_GPIO_Init(GPIOE, &GPIO_InitStruct);
/*Configure GPIO pins : PE14 PE15 */
GPIO_InitStruct.Pin = GPIO_PIN_14|GPIO_PIN_15;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.Alternate = GPIO_AF3_TIM1_COMP1;
HAL_GPIO_Init(GPIOE, &GPIO_InitStruct);
/*Configure GPIO pin : PB10 */
GPIO_InitStruct.Pin = GPIO_PIN_10;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.Alternate = GPIO_AF1_TIM2;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
/*Configure GPIO pins : PB12 PB13 PB15 */
GPIO_InitStruct.Pin = GPIO_PIN_12|GPIO_PIN_13|GPIO_PIN_15;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.Alternate = GPIO_AF13_SAI2;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
/*Configure GPIO pin : PB14 */
GPIO_InitStruct.Pin = GPIO_PIN_14;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.Alternate = GPIO_AF14_TIM15;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
```

```
/*Configure GPIO pins : PD8 PD9 */
GPIO_InitStruct.Pin = GPIO_PIN_8|GPIO_PIN_9;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
GPIO_InitStruct.Alternate = GPIO_AF7_USART3;
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);
/*Configure GPIO pins : PD14 PD15 */
GPIO_InitStruct.Pin = GPIO_PIN_14|GPIO_PIN_15;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.Alternate = GPIO_AF2_TIM4;
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);
/*Configure GPIO pin : PC6 */
GPIO_InitStruct.Pin = GPIO_PIN_6;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.Alternate = GPIO_AF13_SAI2;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
/*Configure GPIO pin : PC7 */
GPIO_InitStruct.Pin = GPIO_PIN_7;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.Alternate = GPIO_AF2_TIM3;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
/*Configure GPIO pins : PC8 PC9 PC10 PC11
                       PC12 */
GPIO_InitStruct.Pin = GPIO_PIN_8|GPIO_PIN_9|GPIO_PIN_10|GPIO_PIN_11
                      |GPIO_PIN_12;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
GPIO_InitStruct.Alternate = GPIO_AF12_SDMMC1;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
/*Configure GPIO pins : PA8 PA10 */
GPIO_InitStruct.Pin = GPIO_PIN_8|GPIO_PIN_10;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
```

```
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
GPIO_InitStruct.Alternate = GPIO_AF10_OTG_FS;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
/*Configure GPIO pin : PA9 */
GPIO_InitStruct.Pin = GPIO_PIN_9;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
/*Configure GPIO pin : PD0 */
GPIO_InitStruct.Pin = GPIO_PIN_0;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
GPIO_InitStruct.Alternate = GPIO_AF9_CAN1;
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);
/*Configure GPIO pin : PD2 */
GPIO_InitStruct.Pin = GPIO_PIN_2;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
GPIO_InitStruct.Alternate = GPIO_AF12_SDMMC1;
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);
/*Configure GPIO pins : PD3 PD4 PD5 PD6 */
GPIO_InitStruct.Pin = GPIO_PIN_3|GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
GPIO_InitStruct.Alternate = GPIO_AF7_USART2;
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);
/*Configure GPIO pins : PB3 PB4 PB5 */
GPIO_InitStruct.Pin = GPIO_PIN_3|GPIO_PIN_4|GPIO_PIN_5;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
GPIO_InitStruct.Alternate = GPIO_AF6_SPI3;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
/*Configure GPIO pin : PE0 */
GPIO_InitStruct.Pin = GPIO_PIN_0;
GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
```

```
 GPIO_InitStruct.Pull = GPIO_NOPULL;
 GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
 GPIO_InitStruct.Alternate = GPIO_AF2_TIM4;
 HAL_GPIO_Init(GPIOE, &GPIO_InitStruct);
 /* USER CODE BEGIN MX_GPIO_Init_2 */
 /* USER CODE END MX_GPIO_Init_2 */
}
/* USER CODE BEGIN 4 */
/* USER CODE END 4 */
/**
 * @brief  This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
 /* USER CODE BEGIN Error_Handler_Debug */
 /* User can add his own implementation to report the HAL error return state */
 __disable_irq();
 while (1)
 {
 }
 /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
 * @brief  Reports the name of the source file and the source line number
 *         where the assert_param error has occurred.
 * @param  file: pointer to the source file name
 * @param  line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
 /* USER CODE BEGIN 6 */
 /* User can add his own implementation to report the file name and line number,
    ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
 /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```